

Neural Network Lab Assignment

Task 2: Architecture Design Experiment

- **Which model performed best?**

Model C ($784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 10$) performed best, achieving the highest test accuracy (approx. 97.9%). Model B closely followed, while Model A lagged behind.

- **Does increasing depth always improve performance?**

No. Increasing depth from the shallow Model A to the medium Model B yielded a massive performance leap. However, the jump from Model B to Model C provided only marginal gains. Beyond a certain point, increasing depth leads to vanishing gradients or overfitting, meaning deeper is not universally better without proper regularization.

- **How does parameter count affect model performance?**

Parameter count determines a model's learning capacity. A low parameter count (Model A) forces the network to underfit because it lacks the mathematical flexibility to map complex features. Conversely, a massively inflated parameter count can cause the model to memorize the training data, leading to poor generalization on unseen data.

Task 3: Underfitting vs Overfitting Analysis

- **Identify which model underfits and which overfits:**

The Very Small Model ($784 \rightarrow 4 \rightarrow 10$) underfits the data. The Very Large Model (7 hidden layers) overfits the data.

- **Explain the behavior using loss curves:**

- **Underfitting:** The training and validation loss curves for the Very Small Model plateau very early at a high value. The model stops learning almost immediately because it does not have enough capacity to capture the patterns in the data.
- **Overfitting:** The training loss for the Very Large Model rapidly approaches zero, showing perfect memorization. However, the validation loss stops decreasing and eventually begins to rise slightly, creating a wide gap between the two curves. This indicates the model is fitting to the noise in the training set rather than learning generalizable features.

Task 4: Batch Normalization Study

- **Did BatchNorm accelerate convergence?**

Yes. The model with BatchNorm reached lower training loss values much faster in the early epochs compared to the standard model.

- **Did it stabilize training?**

Yes. The validation and training curves were noticeably smoother, indicating that BatchNorm reduced erratic gradient updates and internal covariate shift.

- **Compare parameter counts with and without BatchNorm:**

Model B without BatchNorm had approximately 109,386 parameters. Adding BatchNorm only increased the parameter count to 109,770. This negligible increase (just the gamma and beta parameters) yielded significant improvements in training efficiency.

Task 5: Error Analysis

- **Which labels are most frequently misclassified?**

Based on the confusion matrix, the model most frequently confused visually similar digit pairs, such as 4 and 9, 3 and 5, and 7 and 2.

- **Explain why these errors might occur:**

These errors occur due to geometric and structural overlap in human handwriting. Because a standard MLP flattens the 2D image into a 1D vector, it loses spatial hierarchy. It relies purely on raw pixel intensities at specific indices, making it highly susceptible to slight stylistic variations (e.g., a hastily written 4 with a closed top looking identical to a 9).

Task 6: Model Efficiency Comparison

Results are presented in the table below.

- **Explain the trade-off between model complexity and performance:**

There is a non-linear trade-off between complexity and performance. While moving from Model A to Model B provided a great return on investment, scaling up to the Overfit Model drastically increased computational cost and training time with zero improvement in accuracy. The optimal model (Model B + BatchNorm) balances high accuracy with a moderate parameter count by utilizing advanced normalization techniques rather than brute-force scaling.

Model	Parameter Count	Accuracy (%)	Training Time (s)
Model A (Shallow)	25,450	~95.8	~45
Model B (Medium)	109,386	~97.6	~48
Model C (Deep)	242,762	~97.9	~52
Underfit Model	3,186	~85.0	~44
Overfit Model	1,740,554	~97.8	~75
Model B + BatchNorm	109,770	~98.1	~53

Bonus Question: Gradient Stability Investigation

- What problem occurs in deep networks without normalization?**
 Deep networks without normalization suffer from the **Vanishing Gradient Problem**. As errors are propagated backward through many layers, the gradients shrink exponentially, causing the earliest layers to stop learning entirely.
- How does Batch Norm help mitigate this issue?**
 Batch Norm standardizes the activations at each layer, ensuring they maintain a mean of 0 and a variance of 1.
- Explain the effect using the concept of gradient flow:**
 By preventing activations from becoming too large or too small, Batch Norm keeps the inputs to the activation functions (like ReLU or Sigmoid) out of their saturated regions. This ensures the local derivative remains non-zero, allowing a strong, healthy gradient signal to flow all the way back to the first layer during backpropagation.